

Java Backend Interview Prep - Spring Core, Web Layer, and Exception Handling

This document answers the Spring, Spring Boot, controller/web, validation, caching, security, and Spring exception handling topics listed in the source interview sheet.[file:1]

Spring core

1) IoC and DI

Inversion of Control means object creation and wiring are handled by the Spring container instead of application code. **Dependency Injection** is the mechanism used to provide dependencies to beans.[file:1]

Types of DI:

- Constructor injection
- Setter injection
- Field injection (works, but less preferred)

Constructor injection is usually best for immutability and testability.

2) Spring singleton vs Java singleton

A Spring singleton means one bean instance per container, not one instance per JVM classloader like a traditional singleton pattern. A Spring singleton bean is not automatically thread-safe.[file:1]

3) Bean lifecycle

Typical lifecycle:[file:1]

1. Instantiate bean.
2. Inject dependencies.
3. Run aware callbacks.
4. Run `@PostConstruct` / `init` methods.
5. Bean is ready.
6. On shutdown, run `@PreDestroy` / `destroy` methods.

4) Autowiring and multiple implementations

If multiple beans implement the same interface, resolve ambiguity with:

- `@Qualifier`
- `@Primary`
- bean name based injection conventions

```
@Service
class StripePaymentService implements PaymentService {}

@Service
@Primary
class RazorpayPaymentService implements PaymentService {}
```

5) @Configuration and @Bean

`@Configuration` marks a class that declares bean definitions. `@Bean` marks a factory method whose return value becomes a Spring bean.[file:1]

6) @Autowired vs @Inject

`@Autowired` is Spring-specific and supports features like `required=false`. `@Inject` comes from Jakarta/JSR standard and is more portable.

7) ResponseBody vs ResponseEntity

- `@ResponseBody`: serializes return value to response body.
- `ResponseEntity<T>`: gives full control over status, headers, and body.

8) @Controller vs @RestController

`@RestController = @Controller + @ResponseBody`, so every handler returns serialized response body by default.[file:1]

9) @SpringBootApplication

It combines:[file:1]

- `@SpringBootApplication`
- `@EnableAutoConfiguration`
- `@ComponentScan`

10) Component scanning default behavior

If no package is specified, scanning starts from the package of the main application class and its subpackages.[file:1]

Web layer

11) JSON and XML responses

Spring Boot uses content negotiation. JSON is common through Jackson, and XML can be supported by adding the proper XML message converter dependency.[file:1]

12) @RequestParam vs @PathVariable vs @RequestBody

```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping
    public String byStatus(@RequestParam String status) {
        return status;
    }

    @GetMapping("/{id}")
    public Long byId(@PathVariable Long id) {
        return id;
    }

    @PostMapping
    public User create(@RequestBody User user) {
        return user;
    }
}
```

- @RequestParam: query string or form parameter.
- @PathVariable: path segment.
- @RequestBody: deserialize request payload.

13) REST endpoint naming

Best practices:[file:1]

- Use nouns, not verbs.
- Use plural resources like /users.
- Keep paths lowercase and consistent.
- Use HTTP methods to express action.
- Version externally only when needed, e.g. /api/v1/users.

14) HTTP methods and idempotency

- GET: idempotent
- PUT: idempotent
- DELETE: typically idempotent
- POST: not idempotent
- PATCH: not guaranteed idempotent
- OPTIONS: safe/idempotent for metadata

15) File upload and download

Use `MultipartFile` for upload and `ResponseEntity<Resource>` for download.

```
@PostMapping("/upload")
public String upload(@RequestParam MultipartFile file) {
    return file.getOriginalFilename();
}
```

16) CORS

Configure through `WebMvcConfigurer`, `@CrossOrigin`, or Spring Security config depending on scope.[file:1]

17) Request lifecycle

High level flow:[file:1]

DispatcherServlet -> HandlerMapping -> HandlerAdapter -> Controller -> Service -> Response serialization

18) Spring Security + JWT

JWT has three parts: header, payload, signature. Typical flow: authenticate user, issue token, client sends bearer token, filter validates token, security context is populated.[file:1]

19) Rate limiting and throttling

Common approaches:

- API gateway limit
- filter/interceptor level control
- Redis-based token bucket / sliding window
- Resilience4j rate limiter

20) Caching annotations

- `@Cacheable`: read-through, skips method if cache hit exists.
- `@CachePut`: always executes method and updates cache.
- `@CacheEvict`: removes entries.

```
@Cacheable(cacheNames = "users", key = "#id")
public User getUser(Long id) { ... }
```

21) condition and unless in @Cacheable

- `condition`: decide before invocation whether caching applies.
- `unless`: decide after invocation whether result should be cached.

22) Cache stampede prevention

Use TTL jitter, locks/single-flight loading, background refresh, multi-level caching, and Redis/distributed caches in clustered environments.[file:1]

23) DispatcherServlet and async requests

Async request handling can use `Callable`, `DeferredResult`, or `WebAsyncTask`, allowing servlet threads to be released while work completes.[file:1]

24) Validation with @Valid and @Validated

- `@Valid`: standard bean validation, common for nested request bodies.
- `@Validated`: Spring variant that supports validation groups and method-level validation.
[file:1]

```
@PostMapping
public User create(@Valid @RequestBody User user) {
    return user;
}
```

25) Custom validation annotation

Create custom annotation + `ConstraintValidator`.

```
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = StrongPasswordValidator.class)
public @interface StrongPassword {
    String message() default "Weak password";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

26) Nested object and collection validation

Use `@Valid` on nested fields and collection elements.

```
class OrderRequest {
    @Valid
    private CustomerRequest customer;

    @Valid
    private List<ItemRequest> items;
}
```

Spring exception handling

27) @ExceptionHandler

Handles exceptions in a controller or advice class.

```
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    ResponseEntity<String> handle(UserNotFoundException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}
```

28) @ControllerAdvice vs @RestControllerAdvice

`@RestControllerAdvice` is `@ControllerAdvice` + `@ResponseBody`, making it more suitable for REST APIs.[file:1]

29) Validation exception handling

Handle `MethodArgumentNotValidException` centrally and return field-wise error details.

30) ResponseEntityExceptionHandler

Extend it when you want centralized handling with framework-friendly defaults for Spring MVC exception types.[file:1]